

# Thuật toán đường đi ngắn nhất và ứng dụng

Yu Yuanming  
Trường Trung học Beijiang, Quảng Đông

2006

*Nguồn gốc: Thuật toán đường đi ngắn nhất và ứng dụng*

IOI China candidate team papers / 2006 / Yu Yuanming / Yu Yuanming.doc

## Tóm tắt

Bài toán đường đi ngắn nhất là một trong các bài toán cốt lõi của lý thuyết đồ thị. Nó là nền tảng của nhiều thuật toán sâu hơn, đồng thời xuất hiện trong nhiều bối cảnh thực tế. Không ít bài toán bên ngoài không liên quan tới đường đi ngắn nhất vẫn có thể quy về mô hình này. Bài viết giới thiệu các khái niệm cơ bản, các thuật toán thường dùng, phạm vi áp dụng của chúng, rồi minh họa qua một số ví dụ, nhấn mạnh quá trình xây dựng mô hình, suy nghĩ và chứng minh.

**Từ khóa:** đường đi ngắn nhất.

## Contents

|          |                                                     |          |
|----------|-----------------------------------------------------|----------|
| <b>1</b> | <b>Các khái niệm cơ bản</b>                         | <b>2</b> |
| 1.1      | Định nghĩa . . . . .                                | 2        |
| 1.2      | Một số biến thể đơn giản . . . . .                  | 2        |
| 1.3      | Cạnh âm . . . . .                                   | 2        |
| 1.4      | Tính chất quan trọng và kỹ thuật nối lỏng . . . . . | 3        |
| <b>2</b> | <b>Các thuật toán thường dùng</b>                   | <b>4</b> |
| 2.1      | Thuật toán Dijkstra . . . . .                       | 4        |
| 2.2      | Thuật toán Bellman–Ford . . . . .                   | 4        |
| 2.3      | Thuật toán SPFA . . . . .                           | 5        |
| <b>3</b> | <b>Ví dụ ứng dụng</b>                               | <b>6</b> |
| 3.1      | Ví dụ 1: đổi tiền . . . . .                         | 6        |
| 3.2      | Ví dụ 2: đường hai tiêu chí . . . . .               | 6        |
| 3.3      | Ví dụ 3: <i>Layout</i> . . . . .                    | 7        |
| 3.4      | Ví dụ 4: tăng tốc mạng . . . . .                    | 8        |
| <b>4</b> | <b>Tổng kết</b>                                     | <b>9</b> |

# 1 Các khái niệm cơ bản

## 1.1 Định nghĩa

Người đi xe thường muốn tìm hành trình ngắn nhất tới đích. Nếu có một bản đồ ghi khoảng cách giữa các cặp ngã tư, làm thế nào để tìm được hành trình ngắn nhất đó? Một cách hiển nhiên là liệt kê mọi đường đi, tính độ dài của từng đường rồi chọn đường ngắn nhất. Nhưng ngay cả khi bỏ qua các đường đi có chu trình, số tuyến đường vẫn có thể lên tới hàng triệu, và phần lớn trong đó không cần xét tới.

Trong bài toán đường đi ngắn nhất, ta có một đồ thị có hướng có trọng số  $G = (V, E)$ , cùng hàm trọng số

$$w : E \rightarrow \mathbb{R}.$$

Với đường đi  $P = (v_0, v_1, \dots, v_k)$ , trọng số của  $P$  là

$$w(P) = \sum_{i=1}^k w(v_{i-1}, v_i).$$

Trọng số đường đi ngắn nhất từ  $u$  tới  $v$  được định nghĩa là

$$\delta(u, v) = \begin{cases} \min\{w(P) : P \text{ là đường đi từ } u \text{ tới } v\}, & \text{nếu tồn tại đường đi,} \\ +\infty, & \text{nếu không tồn tại đường đi.} \end{cases}$$

Một đường đi từ  $u$  tới  $v$  có trọng số  $\delta(u, v)$  được gọi là đường đi ngắn nhất từ  $u$  tới  $v$ .

Trong ví dụ bản đồ đường bộ, đỉnh biểu diễn ngã tư, cạnh biểu diễn đường nối hai ngã tư, trọng số cạnh biểu diễn độ dài đường. Trọng số không nhất thiết chỉ là khoảng cách; nó cũng có thể biểu diễn thời gian, tiền, tiền phạt, tổn thất, hoặc bất kỳ đại lượng nào cộng tuyến tính dọc theo đường đi.

## 1.2 Một số biến thể đơn giản

**Đường đi ngắn nhất một đích.** Tìm đường đi ngắn nhất từ mọi đỉnh  $v$  tới một đỉnh chỉ định  $u$ . Nếu đảo chiều mọi cạnh, bài toán này trở thành bài toán một nguồn.

**Đường đi ngắn nhất giữa một cặp đỉnh.** Với hai đỉnh  $u, v$ , tìm đường đi ngắn nhất từ  $u$  tới  $v$ . Nếu giải được bài toán một nguồn với nguồn  $u$ , ta cũng giải được bài toán này. Về mặt tiệm cận trong trường hợp xấu nhất, hiện chưa có phương pháp tổng quát tốt hơn thuật toán một nguồn tốt nhất.

**Đường đi ngắn nhất mọi cặp.** Với mọi cặp đỉnh  $u, v$ , tìm đường đi ngắn nhất từ  $u$  tới  $v$ . Có thể chạy thuật toán một nguồn một lần cho mỗi đỉnh làm nguồn.

## 1.3 Cạnh âm

Trong một số bài toán một nguồn, cạnh có thể có trọng số âm. Nếu  $G$  không có chu trình âm đi được từ nguồn  $s$ , thì  $\delta(s, v)$  vẫn được định nghĩa đúng cho mọi đỉnh  $v$ , kể cả khi giá trị đó âm. Nếu tồn tại một chu trình âm đi được từ  $s$ , định nghĩa đường đi ngắn nhất không còn hợp lệ đối với các đỉnh có thể bị ảnh hưởng bởi chu trình đó: ta luôn có thể đi thêm quanh chu trình âm để làm trọng số nhỏ hơn. Khi một đường từ  $s$  tới  $v$  có thể đi qua chu trình âm, ta xem  $\delta(s, v) = -\infty$ .

Tài liệu gốc minh họa bằng một hình có cả cạnh âm và chu trình âm. Nếu chu trình  $\langle c, d, c \rangle$  có trọng số dương, ví dụ  $6 + (-3) = 3 > 0$ , thì nó không phá hủy đường đi ngắn nhất tới  $c, d$ . Ngược lại, nếu chu

trình  $\langle e, f, e \rangle$  có trọng số  $3 + (-6) = -3 < 0$ , thì mọi đỉnh đi được từ chu trình đó, chẳng hạn  $e, f, g$ , có giá trị  $-\infty$ . Một chu trình âm không đi được từ  $s$  không ảnh hưởng tới các giá trị đường đi ngắn nhất từ  $s$ .

Một số thuật toán, chẳng hạn Dijkstra, giả sử mọi cạnh có trọng số không âm. Một số thuật toán khác, chẳng hạn Bellman–Ford, cho phép cạnh âm miễn là không có chu trình âm đi được từ nguồn; nếu có, chúng có thể phát hiện ra sự tồn tại của chu trình âm.

## 1.4 Tính chất quan trọng và kỹ thuật nối lỏng

Các thuật toán trong bài đều dựa trên kỹ thuật **nối lỏng**. Ý tưởng là lặp lại việc giảm cận trên hiện có của trọng số đường đi ngắn nhất tới mỗi đỉnh, cho tới khi cận trên ấy bằng đúng giá trị tối ưu.

**Định lý 1: cấu trúc con tối ưu.** Cho đồ thị có hướng có trọng số  $G = (V, E)$ . Giả sử

$$P = \langle v_1, v_2, \dots, v_k \rangle$$

là một đường đi ngắn nhất từ  $v_1$  tới  $v_k$ . Với mọi  $i, j$  thỏa  $i \leq j$ , đường con

$$P_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$$

là một đường đi ngắn nhất từ  $v_i$  tới  $v_j$ .

**Chứng minh.** Ta phân rã  $P$  thành ba đoạn và có

$$w(P) = w(P_{1i}) + w(P_{ij}) + w(P_{jk}).$$

Nếu tồn tại đường  $P'_{ij}$  từ  $v_i$  tới  $v_j$  với  $w(P'_{ij}) < w(P_{ij})$ , thay đoạn  $P_{ij}$  trong  $P$  bằng  $P'_{ij}$  sẽ cho một đường từ  $v_1$  tới  $v_k$  có trọng số nhỏ hơn  $w(P)$ , mâu thuẫn với giả thiết  $P$  là đường đi ngắn nhất.

Một hệ quả trực tiếp là với mọi cạnh  $(u, v) \in E$ ,

$$\delta(s, v) \leq \delta(s, u) + w(u, v).$$

Quả vậy, đường đi từ  $s$  tới  $u$  ngắn nhất rồi đi thêm cạnh  $(u, v)$  là một ứng viên từ  $s$  tới  $v$ , nên đường tối ưu tới  $v$  không thể dài hơn ứng viên đó.

Với mỗi đỉnh  $v$ , ta dùng  $d[v]$  để lưu cận trên hiện tại của  $\delta(s, v)$ , gọi là **ước lượng đường đi ngắn nhất**. Mảng  $\pi[v]$  lưu đỉnh trước của  $v$  trên đường hiện tại.

```
INITIALIZE-SINGLE-SOURCE(G, s)
  for each vertex v in V[G]
    d[v] := infinity
    pi[v] := NIL
  d[s] := 0
```

```
RELAX(u, v, w)
  if d[v] > d[u] + w(u, v)
    d[v] := d[u] + w(u, v)
    pi[v] := u
```

Mọi thuật toán dưới đây đều gọi thủ tục khởi tạo rồi lặp lại thao tác RELAX. Điểm khác nhau nằm ở số lần nối lỏng từng cạnh và thứ tự nối lỏng các cạnh.

## 2 Các thuật toán thường dùng

Phần này xét Dijkstra, Bellman–Ford và SPFA. Cả ba đều dựa trên nối lỏng, nhưng phạm vi áp dụng và cách cài đặt khác nhau.

### 2.1 Thuật toán Dijkstra

Dijkstra giải bài toán một nguồn trên đồ thị có trọng số cạnh không âm. Trong phần này ta giả sử với mọi  $(u, v) \in E$ ,  $w(u, v) \geq 0$ .

Thuật toán duy trì tập  $S$  gồm các đỉnh đã biết chắc giá trị cuối cùng: với mọi  $v \in S$ ,  $d[v] = \delta(s, v)$ . Mỗi bước chọn đỉnh  $u \in V - S$  có  $d[u]$  nhỏ nhất, đưa  $u$  vào  $S$ , rồi nối lỏng mọi cạnh đi ra từ  $u$ . Nếu dùng hàng đợi ưu tiên  $Q$  chứa các đỉnh thuộc  $V - S$ , thuật toán có dạng sau.

```
DIJKSTRA( $G, w, s$ )
  INITIALIZE-SINGLE-SOURCE( $G, s$ )
   $S :=$  empty set
   $Q := V[G]$ 
  while  $Q$  is not empty
     $u :=$  EXTRACT-MIN( $Q$ )
    insert  $u$  into  $S$ 
    for each vertex  $v$  in Adj[ $u$ ]
      RELAX( $u, v, w$ )
```

Tài liệu gốc có một hình mô tả quá trình chạy Dijkstra: đỉnh đen thuộc  $S$ , đỉnh trắng còn trong  $Q = V - S$ , cạnh được tô biểu diễn giá trị tiền nhiệm  $\pi[v]$ . Vì sau khi khởi tạo không có đỉnh mới được thêm vào  $Q$ , mỗi đỉnh chỉ được lấy ra đúng một lần, vòng lặp chính chạy  $|V|$  lần.

**Định lý 2.** Mỗi khi một đỉnh  $u$  được đưa vào  $S$ , ta có

$$d[u] = \delta(s, u).$$

**Phác chứng.** Thuật toán luôn chọn đỉnh  $u \in V - S$  có ước lượng nhỏ nhất. Do mọi cạnh không âm, một đường còn có khả năng cải thiện  $u$  không thể đi qua một đỉnh chưa thuộc  $S$  mà vẫn cho giá trị nhỏ hơn  $d[u]$ . Các đỉnh trong  $S$  đã được xử lý và các cạnh đi ra từ chúng đã được nối lỏng, nên sau thời điểm chọn  $u$ ,  $d[u]$  không thể giảm nữa.

Nếu cài  $Q$  bằng mảng, mỗi lần EXTRACT-MIN tốn  $O(V)$ , tổng cộng  $O(V^2)$ . Các cạnh trong danh sách kề được xét tổng cộng  $O(E)$  lần, nên toàn bộ là  $O(V^2 + E) = O(V^2)$ .

Với đồ thị thưa, cài  $Q$  bằng heap nhị phân thực dụng hơn. Khi đó EXTRACT-MIN tốn  $O(\log V)$ , xây heap tốn  $O(V)$ , và mỗi lần giảm khóa do nối lỏng tốn  $O(\log V)$ . Độ phức tạp là

$$O((V + E) \log V),$$

thường viết là  $O(E \log V)$  khi  $E \geq V$ . Nếu dùng Fibonacci heap, có thể đạt  $O(V \log V + E)$ , nhưng cấu trúc này phức tạp, hằng số lớn, và ít phù hợp với thi lập trình.

### 2.2 Thuật toán Bellman–Ford

Bellman–Ford giải được bài toán một nguồn trong trường hợp tổng quát hơn: cạnh có thể âm. Nó liên tục giảm các ước lượng  $d[v]$  cho tới khi đạt  $\delta(s, v)$ . Nếu tồn tại chu trình âm đi được từ nguồn, thuật toán báo rằng đường đi ngắn nhất không tồn tại.

```

BELLMAN-FORD( $G, w, s$ )
  INITIALIZE-SINGLE-SOURCE( $G, s$ )
  for  $i := 1$  to  $|V[G]| - 1$ 
    for each edge  $(u, v)$  in  $E[G]$ 
      RELAX( $u, v, w$ )
  for each edge  $(u, v)$  in  $E[G]$ 
    if  $d[v] > d[u] + w(u, v)$ 
      return FALSE
  return TRUE

```

Tư tưởng của Bellman–Ford dựa trên sự kiện: nếu giữa hai đỉnh có đường đi ngắn nhất, thì có một đường đi ngắn nhất không lặp đỉnh, do đó có nhiều nhất  $n - 1$  cạnh. Nếu lặp một đỉnh, ta đã đi qua một chu trình; chu trình dương không có lợi, chu trình âm làm bài toán không có nghiệm hữu hạn, còn chu trình không thì có thể bỏ đi mà không đổi giá trị tối ưu. Theo cấu trúc con tối ưu, đường tốt nhất dùng nhiều nhất  $k$  cạnh có thể thu được từ đường tốt nhất dùng nhiều nhất  $k - 1$  cạnh cộng thêm một cạnh. Vì vậy lặp  $n - 1$  vòng là đủ.

Độ phức tạp của Bellman–Ford là  $O(VE)$ : khởi tạo tốn  $O(V)$ , mỗi trong  $|V| - 1$  vòng xét mọi cạnh tốn  $O(E)$ , và bước kiểm tra chu trình âm cũng tốn  $O(E)$ .

Một tối ưu rất đơn giản là dừng sớm. Nếu trong một vòng quét đầy đủ không có đỉnh nào được cập nhật, các vòng sau cũng không thể cập nhật; khi đó có thể thoát. Trường hợp xấu nhất vẫn là  $O(VE)$ , nhưng trong nhiều dữ liệu thực tế thời gian gần hơn  $O(kE)$ , với  $k$  nhỏ hơn  $|V|$  rất nhiều.

## 2.3 Thuật toán SPFA

SPFA là viết tắt của **Shortest Path Faster Algorithm**. Nó thường dùng khi đồ thị có cạnh âm nên Dijkstra không áp dụng được, còn Bellman–Ford thuần túy quá chậm. Ở đây giả sử không có chu trình âm, tức đường đi ngắn nhất tồn tại.

SPFA vẫn dùng mảng  $d$  và danh sách kề. Ta lập một hàng đợi FIFO chứa các đỉnh cần tối ưu. Mỗi lần lấy đỉnh đầu  $u$ , dùng  $d[u]$  để nối lỏng các cạnh đi ra  $u$ . Nếu ước lượng của một đỉnh  $v$  giảm và  $v$  chưa nằm trong hàng đợi, đưa  $v$  vào cuối hàng. Lặp cho tới khi hàng đợi rỗng.

```

SPFA( $G, w, s$ )
  INITIALIZE-SINGLE-SOURCE( $G, s$ )
   $Q :=$  empty queue
  push  $s$  into  $Q$ 
   $in\_queue[s] :=$  true
  while  $Q$  is not empty
     $u :=$  pop_front( $Q$ )
     $in\_queue[u] :=$  false
    for each edge  $(u, v)$ 
      if  $d[v] > d[u] + w(u, v)$ 
         $d[v] := d[u] + w(u, v)$ 
         $pi[v] := u$ 
        if not  $in\_queue[v]$ 
          push_back( $Q, v$ )
           $in\_queue[v] :=$  true

```

**Định lý 3.** Nếu đường đi ngắn nhất tồn tại, SPFA sẽ tìm được các giá trị tối ưu.

**Chứng minh.** Mỗi lần một đỉnh được đưa vào hàng đợi là do một thao tác nối lỏng làm một giá trị  $d[v]$  giảm. Vì không có chu trình âm, mỗi đỉnh có giá trị đường đi ngắn nhất hữu hạn; quá trình giảm không thể kéo dài vô hạn. Khi hàng đợi rỗng, không còn cạnh nào có thể nối lỏng, nên các ước lượng hiện tại chính là giá trị đường đi ngắn nhất.

Trung bình, SPFA thường được xem là  $O(E)$ . Lý do trực giác là mỗi lần xử lý một đỉnh  $u$  ta xét các cạnh đi ra  $u$ , tốn  $O(\deg^+(u))$ . Nếu số lần vào hàng của các đỉnh trung bình là một hằng số nhỏ, tổng chi phí tỉ lệ với số cạnh. Dĩ nhiên trường hợp xấu nhất vẫn có thể lớn hơn nhiều, nhưng trong nhiều bài thi SPFA nhanh hơn Bellman–Ford tối ưu đơn giản. Cả hai đều là thuật toán hiệu chỉnh nhân: các nhãn tạm thời liên tục được cải thiện, và chỉ ở cuối mới trở thành kết quả. Khác biệt là Bellman–Ford quét toàn bộ các cạnh sau mỗi cập nhật theo vòng, còn SPFA chỉ lan truyền từ những đỉnh vừa bị ảnh hưởng.

### 3 Ví dụ ứng dụng

Mục tiêu của phần này là chỉ ra cách xây mô hình. Trong bài toán thực tế, người ra đề thường không nói sẵn cái gì là đỉnh, cái gì là cạnh; ta phải dựa vào bản chất đề bài để trừu tượng hóa thành một mô hình phù hợp.

#### 3.1 Ví dụ 1: đổi tiền

Có nhiều điểm đổi tiền trong một thành phố. Mỗi điểm chỉ đổi giữa hai loại tiền nhất định; nhiều điểm có thể phục vụ cùng một cặp tiền. Mỗi điểm có tỉ giá và phí riêng. Nếu đổi từ tiền  $A$  sang tiền  $B$ , tỉ giá  $R_{AB}$  cho biết nhận được bao nhiêu đơn vị  $B$  từ một đơn vị  $A$ , còn phí  $C_{AB}$  được trừ bằng tiền nguồn. Chẳng hạn đổi 100 USD với tỉ giá 29.75 và phí 0.39 thì nhận được

$$(100 - 0.39) \cdot 29.75 = 2963.3975.$$

Có  $N \leq 100$  loại tiền, đánh số 1 tới  $N$ . Mỗi điểm đổi tiền được mô tả bởi  $A, B, R_{AB}, C_{AB}, R_{BA}, C_{BA}$ . Nick có một lượng tiền loại  $S$  và muốn biết liệu qua một số lần đổi, cuối cùng vẫn là tiền loại  $S$ , số tiền có thể tăng lên hay không.

Nếu biết lượng tiền lớn nhất có thể đạt được ở mỗi loại tiền, câu trả lời rất rõ: nếu lượng lớn nhất của loại  $S$  không vượt lượng ban đầu thì không thể kiếm lời; ngược lại, quá trình đạt được giá trị lớn hơn là một cách đổi hợp lệ. Việc tìm giá trị lớn nhất phù hợp với bản chất bài toán, vì tiền trước khi đổi càng nhiều thì tiền sau khi đổi càng nhiều.

Ta xem  $N$  loại tiền là  $N$  đỉnh. Mỗi điểm đổi tiền tạo ra hai cạnh có hướng. Nếu từ  $A$  sang  $B$  có tỉ giá  $R_{AB}$  và phí  $C_{AB}$ , thì từ giá trị hiện tại  $x$  ở  $A$ , giá trị ứng viên ở  $B$  là

$$(x - C_{AB})R_{AB}.$$

Bài toán là tìm “đường đi dài nhất” theo phép chuyển này và phát hiện chu trình sinh lời. Vì Dijkstra không xử lý được dạng tương đương cạnh âm này, ta dùng Bellman–Ford hoặc SPFA. Có thể dừng khi không còn giá trị nào cập nhật, hoặc khi giá trị của tiền  $S$  đã vượt giá trị ban đầu. Khi so sánh số thực, nên dùng một ngưỡng nhỏ, ví dụ  $10^{-8}$ .

#### 3.2 Ví dụ 2: đường hai tiêu chí

Đường phố hai chiều có thời gian đi và phí phải trả. Một đường đi có tổng thời gian và tổng phí. Với cùng điểm xuất phát và đích, nếu đường  $A$  vừa tốn ít thời gian hơn vừa tốn ít phí hơn đường  $B$ , ta nói  $A$  tốt hơn  $B$ . Một đường được gọi là tối ưu nếu không có đường khác tốt hơn nó. Cho mạng giao thông,

điểm đầu và điểm cuối, hãy đếm số đường tối ưu. Số thành phố không quá 100, số cạnh không quá 300, thời gian và phí trên mỗi cạnh không quá 100.

Khó khăn là nhãn không còn một chiều mà gồm hai tiêu chí, nên không có thứ tự toàn phần đơn giản. Ta tách trạng thái bằng phí: đặt  $d[i, c]$  là thời gian nhỏ nhất để đi từ nguồn tới thành phố  $i$  với tổng phí đúng  $c$ . Sau khi tách như vậy, ta lại có thể dùng thuật toán đường đi ngắn nhất.

Có hai hướng chính. Hướng thứ nhất là thuật toán gán nhãn kiểu Dijkstra: biến các nhãn tạm thời “cực tiểu” thành nhãn vĩnh viễn. Ở bài này, một nhãn có thể vĩnh viễn nếu không thể nhận được từ nhãn vĩnh viễn khác với phí không lớn hơn và thời gian không lớn hơn. Giả sử cận thời gian là  $t$ , cận phí là  $c$ , số thành phố là  $n$ , số cạnh là  $m$ . Số nhãn tại mỗi đỉnh là  $O(nc)$ , tổng số nhãn là  $O(n^2c)$ , và mỗi cạnh được xét  $O(nc)$  lần. Nếu dùng heap để duy trì các nhãn theo phí và theo đỉnh, độ phức tạp có thể viết là

$$O(n^2c + mnc \log n \log c).$$

Hướng thứ hai là dùng SPFA trên đồ thị trạng thái. Trong trường hợp xấu nhất, cận phí có thể là  $100 \cdot 100 = 10000$ , mỗi đỉnh bị tách thành nhiều trạng thái, và số cạnh trạng thái có thể lên tới vài triệu. Trường hợp xấu nhất lý thuyết rất lớn, nhưng dữ liệu thực tế của bài này đặc biệt: cận phí tối đa hiếm khi đạt tới 10000, và các trạng thái tách ra từ cùng một đỉnh khá độc lập. Với hàng đợi vòng, cài đặt SPFA đơn giản, tiết kiệm bộ nhớ và chạy rất nhanh trong thực nghiệm.

### 3.3 Ví dụ 3: *Layout*

Bài *Layout* nói về  $N$  con bò đứng trên một đường thẳng theo đúng thứ tự số hiệu  $1, \dots, N$ . Nhiều con có thể đứng cùng tọa độ. Một số cặp bò thích nhau và muốn khoảng cách không vượt quá  $L$ ; một số cặp ghét nhau và muốn khoảng cách không nhỏ hơn  $D$ . Có  $ML, MD \leq 10000$  ràng buộc. Cần in  $-1$  nếu không có cách xếp,  $-2$  nếu khoảng cách giữa bò 1 và bò  $N$  có thể lớn tùy ý, ngược lại in khoảng cách lớn nhất có thể giữa chúng.

Khi bài toán phức tạp, nên lùi lại xét phiên bản dễ hơn: chỉ cần tìm một cách xếp hợp lệ. Gọi  $D[i]$  là khoảng cách từ bò  $i$  tới bò 1. Vì thứ tự theo số hiệu được giữ nguyên, với  $1 \leq i < N$  ta có

$$D[i+1] - D[i] \geq 0.$$

Với ràng buộc thích nhau  $(i, j, k)$ , giả sử  $i \leq j$ , ta có

$$D[j] - D[i] \leq k.$$

Với ràng buộc ghét nhau  $(i, j, k)$ , giả sử  $i \leq j$ , ta có

$$D[j] - D[i] \geq k.$$

Đây là một **hệ ràng buộc hiệu**. Viết lại dưới dạng chuẩn:

$$D[i] \leq D[i+1], \quad D[j] \leq D[i] + k, \quad D[i] \leq D[j] - k.$$

Dạng này giống bất đẳng thức đường đi ngắn nhất

$$d(v) \leq d(u) + w(u, v).$$

Ta dựng đồ thị  $G = (V, E)$  với  $V = \{v_1, \dots, v_n\}$ . Với hai chỉ số kề nhau, thêm cạnh  $v_{i+1} \rightarrow v_i$  trọng số 0. Với mỗi ràng buộc thích nhau  $(a_i, b_i, d_i)$ , đổi chỗ để  $a_i < b_i$ , rồi thêm cạnh  $v_{a_i} \rightarrow v_{b_i}$  trọng số  $d_i$ . Với mỗi ràng buộc ghét nhau, thêm cạnh  $v_{b_i} \rightarrow v_{a_i}$  trọng số  $-d_i$ .

**Định lý 5.** Bài toán có nghiệm khi và chỉ khi đồ thị  $G$  không có chu trình âm.

**Chứng minh.** Nếu  $G$  không có chu trình âm, ta tính được độ dài đường đi ngắn nhất từ  $v_1$  tới các đỉnh. Với mọi cạnh  $u \rightarrow v$ , tính chất đường đi ngắn nhất cho  $d(v) \leq d(u) + w(u, v)$ , đúng chính xác với các ràng buộc đã dựng. Do đó dãy tọa độ thỏa mãn tồn tại. Nếu  $G$  có chu trình âm, các ước lượng có thể bị giảm mãi; tương đương luôn tồn tại ít nhất một ràng buộc bị vi phạm, nên không có nghiệm.

Để giải bài gốc, đặt mọi ước lượng ban đầu là một số đủ lớn, riêng  $v_1$  là 0, rồi chạy Bellman–Ford. Nếu có chu trình âm, in  $-1$ . Nếu sau khi chạy,  $v_N$  vẫn có giá trị đủ lớn, khoảng cách giữa 1 và  $N$  có thể lớn tùy ý, in  $-2$ . Ngược lại, in giá trị của  $v_N$ . Số “đủ lớn” chỉ cần lớn hơn tích của trọng số cạnh lớn nhất và số đỉnh, vì trong một đường không chu trình mỗi đỉnh xuất hiện nhiều nhất một lần.

Nếu  $v_N$  vẫn bằng số đủ lớn, các ràng buộc không ép được một cận trên hữu hạn cho khoảng cách 1 tới  $N$ , nên có thể đẩy nó lên tùy ý. Nếu Bellman–Ford cho một giá trị hữu hạn  $D[N]$ , giá trị này chính là khoảng cách lớn nhất hợp lệ: mọi nghiệm hợp lệ đều thỏa các bất đẳng thức cạnh nên không thể vượt qua cận ngắn nhất thu được, còn chính các giá trị  $D[i]$  lại tạo thành một nghiệm hợp lệ.

Độ phức tạp xấu nhất là

$$O((ML + MD)N),$$

đủ cho  $ML, MD \leq 10000$  và  $N \leq 1000$ .

### 3.4 Ví dụ 4: tăng tốc mạng

Một mạng máy tính có  $n \leq 50$  máy, nối với nhau bằng các đường truyền có thời gian truyền khác nhau. Có  $m \leq 10$  thiết bị tăng tốc; mỗi thiết bị đặt lên một cạnh làm thời gian trên cạnh đó giảm một nửa. Nhiều thiết bị có thể đặt trên cùng một cạnh, tác dụng nhân lên: hai thiết bị còn 1/4, ba thiết bị còn 1/8. Cần đặt thiết bị để thời gian truyền từ máy 1 tới máy  $n$  nhỏ nhất.

Ví dụ vào:

```
5 2
0 34 24 0 0
34 0 10 12 0
24 10 0 16 20
0 12 16 0 30
0 0 20 30 0
```

Một đáp án tối ưu là thời gian 22.00, đặt thiết bị lên các cạnh (1, 3) và (3, 5).

Nếu chạy Dijkstra trực tiếp trên đồ thị ban đầu, ta chỉ nhận được đường ngắn nhất khi chưa dùng thiết bị. Cách tham lam tăng dần cũng sai: phương án tối ưu với  $m - 1$  thiết bị không nhất thiết là một phần của phương án tối ưu với  $m$  thiết bị.

Điểm quan trọng là  $m$  rất nhỏ. Ta loại bỏ sự khác biệt giữa “đã dùng bao nhiêu thiết bị” bằng cách tách lớp. Với mỗi đỉnh gốc  $v_i$ , tạo  $m + 1$  đỉnh

$$v_{i,0}, v_{i,1}, \dots, v_{i,m},$$

trong đó chỉ số lớp là số thiết bị đã dùng. Với mỗi cạnh vô hướng gốc  $(v_i, v_j)$  trọng số  $w(v_i, v_j)$ , và với mọi  $0 \leq p \leq q \leq m$ , thêm hai cạnh có hướng

$$v_{i,p} \rightarrow v_{j,q}, \quad v_{j,p} \rightarrow v_{i,q}.$$

Nếu đi từ lớp  $p$  lên lớp  $q$ , nghĩa là dùng thêm  $q - p$  thiết bị trên cạnh đó, trọng số cạnh mới là

$$\frac{w(v_i, v_j)}{2^{q-p}}.$$



Không có cạnh từ lớp lớn xuống lớp nhỏ, đúng với việc một thiết bị đã dùng không thể lấy lại.

Sau khi dựng đồ thị mới, chỉ cần chạy Dijkstra từ  $v_{1,0}$  tới  $v_{n,m}$ . Giá trị thu được là thời gian truyền nhỏ nhất sau khi đặt đủ  $m$  thiết bị.

## 4 Tổng kết

Bài viết trước hết trình bày các khái niệm của đường đi ngắn nhất, làm nền cho ba thuật toán thường dùng. Dijkstra rất nhanh nhưng chỉ áp dụng trực tiếp cho đồ thị không có cạnh âm. Bellman–Ford áp dụng rộng hơn và phát hiện được chu trình âm, nhưng thường chậm. SPFA kết hợp nhiều ưu điểm: cài đặt đơn giản, thường nhanh, và xử lý được đồ thị có cạnh âm nếu đường đi ngắn nhất tồn tại.

Trong đồ thị không có cạnh âm, nhất là đồ thị dày, Dijkstra là lựa chọn tự nhiên; với đồ thị thưa, Dijkstra dùng heap nhị phân hoặc SPFA đều đáng cân nhắc. Nếu có cạnh âm, Bellman–Ford và SPFA là hai lựa chọn chính, trong đó SPFA thường là lựa chọn thực dụng khi đồ thị lớn hoặc dày. Điều quan trọng là cân bằng giữa độ phức tạp thời gian, bộ nhớ và độ phức tạp lập trình.

Với phần lớn bài đường đi ngắn nhất, thuật toán kinh điển đã đủ. Khó khăn thường nằm ở mô hình hóa: hiểu bản chất đề bài, biến nó thành đỉnh, cạnh và trọng số phù hợp. Hai câu kết của tác giả có thể tóm lại như sau: mô hình có thể biến hóa, nhưng tư tưởng cốt lõi không đổi; hãy nắm chắc bản chất đường đi ngắn nhất, rồi chọn thuật toán phù hợp với dữ liệu cụ thể.

## Lời cảm ơn

Tác giả cảm ơn thầy Huang Yeting đã hướng dẫn, góp ý và chỉ ra các thiếu sót của bài viết; đồng thời cảm ơn Zou Yuhua và Jiang Hongsheng đã giúp đỡ trong quá trình viết bài.

## Tài liệu tham khảo

1. Wu Wenhua, Wang Jiande, *Phân tích thuật toán và thiết kế chương trình thực dụng*, Nhà xuất bản Công nghiệp Điện tử.
2. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa.
3. Wang Jiande, Zhou Yongji, *Con đường huy chương vàng: tin học trung học*, Nhà xuất bản Đại học Sư phạm Thiểm Tây.
4. Thomas H. Cormen và cộng sự, *Introduction to Algorithms*, The MIT Press.